

Document number	P2497R0
Date	2023-01-24
Audience	LEWG
Reply-to	Jonathan Wakely <cxx@kayari.org>

Testing for success or failure of `<charconv>` functions

- [Introduction](#)
- [Background](#)
- [Proposed solution](#)
- [Alternatives](#)
 - [Do nothing](#)
 - [Overloaded operator!\(`errc`\).](#)
 - [Named function `ok\(errc\)`.](#)
 - [Named member functions](#)
 - [Something something `std::expected`](#)
- [Proposed wording](#)

Introduction

Every time I use `to_chars` or `from_chars` I find checking `res.ec == std::errc{}` really clunky. Doing `!static_cast<bool>(res.ec)` isn't any better. For `std::error_code` we have an explicit conversion to `bool`, so you can do `if (ec)` but that doesn't work for the low-level `std::errc`.

Can we improve it?

This proposal addresses [C++23 NB comment GB-083](#).

Background

When `to_chars` and `from_chars` were first added to the C++17 draft, by [P0067R5 \("Elementary string conversions"\)](#), errors were reported via a data member, `ec`, of type `std::error_code`.

To resolve [LWG 2955 \("to_chars / from_chars depend on std::string"\)](#), the type of the `ec` member was changed to `std::errc` by [P0682R1 \("Repairing elementary string conversions"\)](#). As noted in the paper, "the usage pattern of the functions deteriorates". To check for errors requires comparing against `std::errc{}` (because there is no `errc` constant with that value) or explicitly casting to `bool`:

```
auto [ptr, ec] = std::to_chars(p, last, 42);
if (ec == std::errc{}) // or !static_cast<bool>(ec)
...
```

Neither option is very readable in my opinion. Unfortunately, the P0682 API change made those functions less ergonomic, and that was never repaired. I would like to address that now.

Proposed solution

When I wrote [C++23 NB comment GB-083](#) on this topic I suggested overloading `operator!` for `std::errc`. That would allow testing the `to_chars_result::ec` member directly. However, that would be novel for standard library enumeration types, and the conversion would be valid for any `std::errc` value, which is not actually necessary if all we want is to improve the `to_chars` and `from_chars` APIs.

The other downside of overloading `operator!` for `errc` is that you would be able to check for an error easily, with `if (!ec)`. But to check for no error you would need to do `if (!!ec)`. This asymmetry would be annoying.

Additionally, testing "error is false" seems less direct and less expressive than "result is true". I want to know if the function was successful, not if the result's error is false.

In Kona, LEWG expressed a preference (which I agreed with) for adding `explicit operator bool()` to `to_chars_result` and `from_chars_result`. That makes it much more convenient to check for a successful result:

```
if (auto res = std::to_chars(p, last, 42))
    ...
```

It doesn't help when structured bindings are used though, because in that case there is no `to_chars_result` that can be converted to `bool`:

```
auto [ptr, ec] = std::to_chars(p, last, 42);
if (???)
```

This would mean either you have to choose between using structured bindings or using the conversion operator, or you have to have an extra variable to be able to use both:

```
if (auto res = std::to_chars(p, last, 42)
{
    auto [ptr, _] = res;
    ...
}
```

This is unfortunate, but I think it's still an improvement on what we have in C++20 today.

Alternatives

Do nothing

We've lived with it since C++17, so we could reject the NB comment and do nothing. Personally, I haven't been happy living with it since C++17 so I don't want to continue with that.

Overloaded operator!(errc)

Drawbacks discussed above.

Named function ok(errc)

A named function for checking an `errc` value, such as `bool ok(errc)`, has few benefits over the overloaded operator, except for not requiring `!!` to test for truthiness.

Named member functions

Instead of adding `to_chars_result::operator bool() const` we could add `bool to_chars_result::ok() const`. This would be consistent with the calendar types in `<chrono>`, so isn't entirely novel. However, those are

value types and they still contain a value even if it isn't "OK". It doesn't really make sense to convert a `chrono::month` to true or false, so a named function to check for a valid value is appropriate there.

For `to_chars_result` and `from_chars_result` we're talking about results of a function, and there's a much more obvious mapping to a boolean value. If the function was successful, the result should be "true", and if the function failed, the result should be "false". I don't think a named function has any advantage here.

Something something `std::expected`

It might seem like we could use `std::expected<char*, std::errc>` for `to_chars_result`, or at least add a conversion to that `std::expected` type. If we'd had `std::expected` in C++17 maybe that's what we'd have done. However it's not true that the result is *either* a pointer *or* an error. The `ptr` member has a defined value in the error cases, and for `from_chars` it has a different value depending on the specific type of error.

We could still choose to add a conversion to `std::expected` which would work for the most common cases (certainly for the success case), and if you care about the specific value of `ptr` in the failure cases, just don't use the conversion. That might be something to consider as an extension later, but I don't think it is a *better* choice than the bool conversion being proposed here.

Proposed wording

This wording is relative to [N4917](#).

Update the value of the `__cpp_lib_to_chars` macro in [version.syn].

Modify [charconv.syn] as indicated:

```
// 22.13.2, primitive numerical output conversion
struct to_chars_result {
    char* ptr;
    errc ec;
    friend bool operator==(const to_chars_result&, const to_chars_result&) = default;
    constexpr explicit operator bool() const noexcept { return ec == errc{}; }
};

...

// 22.13.3, primitive numerical input conversion
struct from_chars_result {
    const char* ptr;
    errc ec;
    friend bool operator==(const from_chars_result&, const from_chars_result&) = default;
    constexpr explicit operator bool() const noexcept { return ec == errc{}; }
};
```